A3-PS013-TC081

Incident Timeline and Evidence Collection for Small Teams

Ishaan Seal, Aishik Ghosh (Team NexGen)

Mentor: Prof. Sayan Das

St. Xavier's University, Kolkata · AIORI Hackathon 3

October 12, 2026

Problem Description

During a breach, small teams lose critical hours rebuilding what happened, and in what order, from scattered and inconsistent logs.

- **Scattered evidence:** Linux auth/syslog, web-server access logs and appliance logs each use their own format.
- **Misaligned clocks:** classic syslog lines have no year and no time zone; hosts run in different zones and clocks drift, so events appear out of order.
- **Fragile integrity:** logs copied and opened by hand leave no proof that the evidence is unchanged.
- **Silent gaps:** lines that do not match the analyst's search patterns are never looked at.
- **Inconsistent results:** the timeline depends on who builds it and how much time they have.
- **Hard deadline:** CERT-In requires cyber incidents to be reported within 6 hours of noticing them [8].

> **Goal:** a lightweight tool that collects host and application logs, preserves them as evidence, normalises every timestamp to UTC and produces a searchable timeline and an exportable report, quickly and consistently.

Background

A trustworthy incident timeline needs two things: evidence that provably has not changed, and timestamps on one consistent clock.

Digital forensics process (NIST SP 800-86 [1])

- **Collection → Examination → Analysis → Reporting.**
- Integrity is shown with cryptographic hashes taken at collection; analysis works on copies, never on the originals.
- Every hand-off is recorded as a chain of custody (RFC 3227 [2]).

Why log time is hard

- Classic syslog (RFC 3164 [3]) carries no year and no zone:
  `Oct 12 13:37:01 web01 sshd[812]: ...`
- Web access logs carry an offset:
  `[12/Oct/2026:13:37:01 +0530]`
- Unsynchronised hosts drift apart (NTP, RFC 5905 [6]); a few minutes of skew can reverse cause and effect.

Indian context: the CERT-In Directions of April 2022 [8] require reporting within 6 hours, keeping logs for 180 days and synchronising clocks to NIC or NPL time servers. Many small teams have no SIEM and work from raw log files on a laptop.

Out of scope (as per the brief): live endpoint takeover, malware removal and any claim of forensic certification.

Literature & RFC Review

Source	Key idea	How we use it
NIST SP 800-86 [1]	Forensic process; hash to verify integrity; work on copies	Pipeline stages; SHA-256 at collection
RFC 3227 [2]	Minimise changes; note clock vs. UTC; chain of custody	Read-only originals; custody log; clock offset
RFC 3164 / 5424 [3, 4]	Legacy syslog (no year or zone) vs. structured syslog	Auth/syslog parser; year and zone inference
RFC 3339 [5]	Unambiguous Internet timestamps	Output format, e.g. `2026-10-12T08:07:01Z`
RFC 5905 [6]	NTP clock synchronisation and offset	Model for cross-source skew estimation
FIPS 180-4 [7]	SHA-256 secure hash	Evidence hashing and re-verification
Apache log docs [9]	Common / Combined Log Format	Access-log parser
Plaso / log2timeline [10]	Super timelines from many forensic artefacts	Shows the value of timelines; heavy for small teams

Gap: existing options are either heavyweight (Plaso, SIEM platforms) or manual (grep and spreadsheets). We found none that combines evidence hashing, per-line traceability and automatic clock-skew estimation in a laptop-sized tool.

Proposed Solution

A lightweight Python tool that works offline and turns raw logs into a verified, searchable UTC timeline and an evidence-backed incident report.

1 Collect & preserve

Copy each log into a read-only evidence store, compute its SHA-256 and write a chain-of-custody record (who, when, source, hash, size).

2 Parse two formats

Linux auth.log / syslog (classic and ISO timestamps) and Apache / Nginx access logs. Unparsed lines are kept and reported, never dropped.

3 Normalise time

Convert to UTC, infer the missing year (with Dec → Jan rollover), apply each source's time zone, then estimate and correct clock skew.

4 Normalised copies

One JSON record per event that keeps the file hash, the line number and the raw original line, next to the untouched original.

5 Search the timeline

Merge all sources in time order; filter by time window, IP address, user or keyword in a local web viewer.

6 Export the report

Incident summary in which every statement cites its log line and file hash, plus a section on parsing gaps.

Validation: three synthetic lab scenarios with known ground truth (SSH brute force → login → sudo; web attack → web shell; data exfiltration) and a triage-time study against a manual baseline. **Stack:** Python 3.11+ (standard library first), pytest, Streamlit; runs on Windows and Linux.

Optimization Proposed

Before and after: what changes for a small team during triage.

Aspect	Before (manual)	After (our tool)
Evidence integrity	Ad-hoc copies, no hashes, accidental edits possible	SHA-256 at collection, read-only originals, custody log, one-command re-verification
Time zones & year	Mental arithmetic; missing year guessed	Every event in UTC; year inferred; zone recorded per source
Clock skew ★	Usually unnoticed, so events end up in the wrong order	Estimated by matching the same event in two logs; the offset is reported and applied
Unparsed lines ★	Silently skipped	Counted and documented as parsing gaps; AI suggests a parsing rule, a person approves it
Traceability ★	Copy-paste and screenshots in the report	Every report statement links to a file hash and line number
Triage time ★	Slow and varies by analyst	Measured in a study with classmates against a manual baseline, framed by CERT-In's 6-hour window

★ our differentiators. An approved AI suggestion becomes an ordinary, unit-tested parser rule, so parsing stays deterministic and reproducible; nothing is sent to an AI service unless the analyst opts in.

Solution Architecture & Design

Component view: a pipeline of small modules, each tested on its own (arrows show data flow).

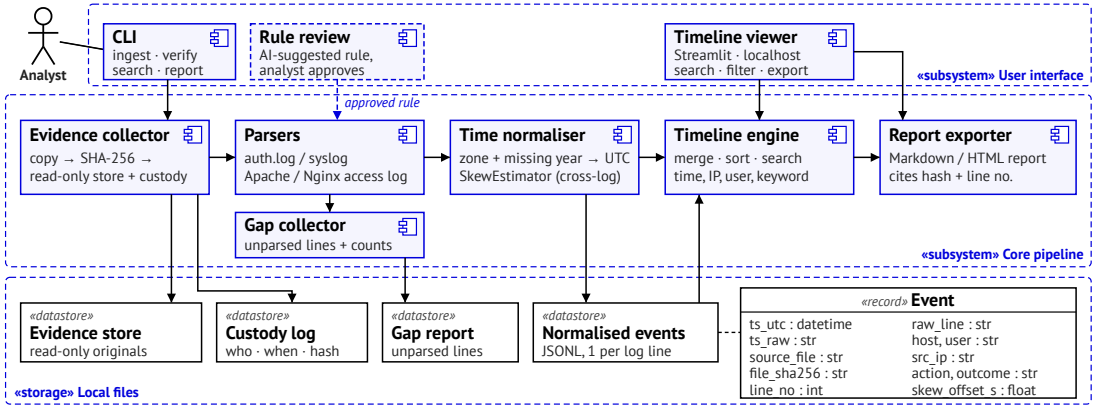


Figure: System Architecture UML Diagram

Data Flow Diagram

Level-1 data flow: every store stays traceable to the original, hashed evidence.

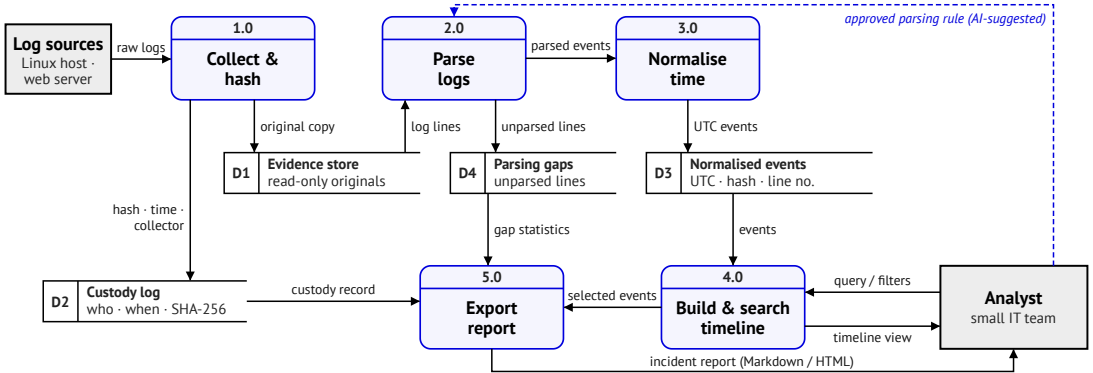


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Milestones follow the AIORI-3 schedule; the tool is built in small, reviewed and tested increments.

Milestones

- **12–14 Oct** Level 1: repository, this proposal, team video
- **15–18 Oct** Evidence store, SHA-256 and custody log; auth.log and access-log parsers; unit tests
- **19–25 Oct** UTC normalisation and skew estimation; timeline search and viewer; scenarios 1–2
- **26–31 Oct** Report export; parsing-gap analysis and AI rule assistant; scenario 3; triage-time study; code and demo submitted
- **24–30 Nov** Regional round (if shortlisted): hardening, extra log formats, usability fixes
- **17 Dec** National finals, IIT Jammu (if selected)

Methodology

- Feature branches and pull requests, each reviewed by the other teammate
- pytest unit tests for every module; sample logs pinned so hashes match on Windows and Linux
- Each lab scenario has a written ground truth that doubles as an end-to-end test
- Triage study: classmates answer the same questions with and without the tool; we compare time and accuracy
- Mentor feedback is folded into the next milestone

Deployment: runs offline on a laptop (only the optional AI rule assistant goes online); `pip install -r requirements.txt`; command-line tool plus a local web viewer; Windows and Linux.

Bibliography I

[1] K. Kent, S. Chevalier, T. Grance and H. Dang. *Guide to Integrating Forensic Techniques into Incident Response*. NIST Special Publication 800-86, 2006.

[2] D. Brezinski and T. Killalea. *Guidelines for Evidence Collection and Archiving*. RFC 3227 (BCP 55), IETF, 2002.

[3] C. Lonvick. *The BSD syslog Protocol*. RFC 3164, IETF, 2001.

[4] R. Gerhards. *The Syslog Protocol*. RFC 5424, IETF, 2009.

[5] G. Klyne and C. Newman. *Date and Time on the Internet: Timestamps*. RFC 3339, IETF, 2002.

[6] D. Mills, J. Martin, J. Burbank and W. Kasch. *Network Time Protocol Version 4: Protocol and Algorithms Specification*. RFC 5905, IETF, 2010.

[7] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015.

[8] CERT-In. *Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents for Safe & Trusted Internet*. No. 20(3)/2022-CERT-In, 28 April 2022.

[9] The Apache Software Foundation. *Log Files*. Apache HTTP Server Version 2.4 Documentation.

[10] The log2timeline project. *Plaso (log2timeline) documentation*. https://plaso.readthedocs.io

Thank You for Your Attention!